

SmartHome Controller Version 1.5

1. Beschreibung

Funktionalität:

- **DHT11** Temperatur und Luftfeuchtigkeit messen
- **HC-SR04** Abstand messen + Ampel (Grün/Gelb/Rot) für Parkhilfe
- **Alarm Button (A3)** steuert virtuellen Alarm:
 - Anzeige auf LCD Zeile 4: „Virtueller Alarm AN/AUS“ (max 5 s)
 - Blinksequenz LEDs bei Alarm AN/AUS bleibt erhalten
- **Water Sensor (A0, analog):**
 - Wasser erkannt → Zeile 4: “WASSER ALARM!”
 - Danach 3 s Anzeige des Wasserlevels als Balken (■)
 - Wiederholt, bis kein Wasser mehr
 - Rote LED blinkt bei Alarm
- **LCD Zeile 4:**
 - Rollierende 25 kurze Statusmeldungen, wenn kein Wasser
 - Zeigt Alarmmeldungen bei Button oder Wasser
- **Nicht-blockierende Logik** für alle Sensoren und Anzeigen

Besonderheiten Version 1.5:

- Analoger Water-Sensor stabiler als digitaler Ausgang
- Balkenanzeige proportional zum Sensorwert (0–1023 → 0–20 Zeichen)
- Alle Funktionen laufen simultan, nicht-blockierend

2. Pinbelegung

Bauteil	Pin
DHT11	2
Ultraschall TRIG	8
Ultraschall ECHO	9
LED Grün	11
LED Gelb	12
LED Rot	13
Alarm Button	A3
Water Sensor AO	A0
LCD I2C	SDA/SCL

3. Materialliste

- 1x Arduino Uno / Nano
- 1x DHT11 Sensor
- 1x HC-SR04 Ultraschallsensor
- 1x Water Sensor Modul (AO)
- 3x LEDs (Grün, Gelb, Rot) ggf. schon in Ampel Modul
- 1x Taster / Button für Alarm
- 1x 20x4 I2C LCD Modul
- Widerstände, Jumper Kabel, Breadboard
- 5V Stromversorgung, ggf. externe Versorgung für Sensoren/LEDs

4. Sketch Version 1.5

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>

// Version 1.5
String VERSION = "SmartHome v1.5";

#define DHTPIN 2
#define DHTTYPE DHT11
DHT dht(DHTPIN,DHTTYPE);

LiquidCrystal_I2C lcd(0x3F,20,4);

#define TRIG_PIN 8
#define ECHO_PIN 9

#define LED_GREEN 11
#define LED_YELLOW 12
#define LED_RED 13

#define ALARM_PIN A3
bool alarmOn=false;
unsigned long lastDebounce=0;
const unsigned long debounceTime=50;

#define WATER_ANALOG A0
int waterValue=0;
bool waterAlarm=false;
const int WATER_THRESHOLD=500;

unsigned long lastBlink=0;
bool redBlinkState=false;

bool showTemp=true;
unsigned long lastSwitchMillis=0;
const unsigned long SWITCH_INTERVAL=3000;
float lastTemp=0;
float lastHum=0;

unsigned long lastDistanceMillis=0;
const unsigned long DISTANCE_INTERVAL=200;
float lastDistance=0;

const int msgCount=25;
String messages[msgCount]={"System aktiv","Sensor online","Alles stabil","Bitte warten","Netzwerk ok","Messung laeuft","SmartHome aktiv","Technik bereit","Status normal","Energie ok","Kontrolle aktiv","Umgebung ok","System prueft","Sensor aktiv","Smart Mode","Daten
```

```

    stabil", "Technik laeuft", "Monitoring", "System online", "Ready", "Betrieb
    ok", "Alles gut", "Live Daten", "System check", "Standby Mode"};
    int msgIndex=0;
    unsigned long lastMsgChange=0;
    const unsigned long MSG_INTERVAL=15000;

    bool showAlarmMessage=false;
    unsigned long alarmMsgStart=0;

    bool alarmSequenceRunning=false;
    int sequenceStep=0;
    unsigned long sequenceStart=0;

    // Wasseranzeige
    unsigned long lastWaterDisplay=0;
    const unsigned long WATER_DISPLAY_INTERVAL=3000;
    bool showWaterLevel=false;

    void setup(){
        Serial.begin(9600);
        dht.begin();
        pinMode(TRIG_PIN, OUTPUT);
        pinMode(ECHO_PIN, INPUT);
        pinMode(LED_GREEN, OUTPUT);
        pinMode(LED_YELLOW, OUTPUT);
        pinMode(LED_RED, OUTPUT);
        pinMode(ALARM_PIN, INPUT_PULLUP);
        pinMode(WATER_ANALOG, INPUT);
        lcd.init();
        lcd.backlight();
        lcd.setCursor(0,0);
        lcd.print(" SmartHome Lienhard ");
        lcd.setCursor(0,3);
        lcd.print(messages[0]);
    }

    void loop(){
        unsigned long now=millis();

        // Wasserwert
        waterValue=analogRead(WATER_ANALOG);
        waterAlarm=(waterValue>=WATER_THRESHOLD);

        if(waterAlarm){
            if(now-lastWaterDisplay>=WATER_DISPLAY_INTERVAL){
                lastWaterDisplay=now;
                showWaterLevel=!showWaterLevel;
            }

            lcd.setCursor(0,3);

```

```

lcd.print("                                ");

if(showWaterLevel){
    int level=map(waterValue,0,1023,0,20);
    String bar="";
    for(int i=0;i<level;i++) bar+="█";
    for(int i=level;i<20;i++) bar+=" ";
    lcd.setCursor(0,3);
    lcd.print("Wasser: "+bar);
} else {
    lcd.setCursor(0,3);
    lcd.print("WASSER ALARM!          ");
}

if(now-lastBlink>300){
    lastBlink=now;
    redBlinkState=!redBlinkState;
    digitalWrite(LED_RED,redBlinkState);
}
}

// Alarm Button
bool switchState=(digitalRead(ALARM_PIN)==LOW);
if(switchState!=alarmOn && now-lastDebounce>debounceTime){
    lastDebounce=now;
    alarmOn=switchState;
    showAlarmMessage=true;
    alarmMsgStart=now;
    alarmSequenceRunning=true;
    sequenceStep=0;
    sequenceStart=now;
    lcd.setCursor(0,3);
    lcd.print("                                ");
    lcd.setCursor(0,3);
    if(alarmOn) lcd.print("Virtueller Alarm AN");
    else lcd.print("Virtueller Alarm AUS");
}

// Blinksequenz Alarm
if(alarmSequenceRunning){
    unsigned long elapsed=now-sequenceStart;
    int shortDur=200;
    int longDur=800;
    if(alarmOn){
        if(sequenceStep<3){
            if(elapsed<shortDur) digitalWrite(LED_RED,HIGH);
            else if(elapsed<shortDur*2) digitalWrite(LED_RED,LOW);
            else {sequenceStep++; sequenceStart=now;}
        }
        else if(sequenceStep==3){
            if(elapsed<longDur) digitalWrite(LED_YELLOW,HIGH);

```

```

        else{digitalWrite(LED_YELLOW,LOW); alarmSequenceRunning=false;}
    }
} else {
    if(sequenceStep<3){
        if(elapsed<shortDur) digitalWrite(LED_YELLOW,HIGH);
        else if(elapsed<shortDur*2) digitalWrite(LED_YELLOW,LOW);
        else {sequenceStep++; sequenceStart=now;}
    }
    else if(sequenceStep==3){
        if(elapsed<longDur) digitalWrite(LED_RED,HIGH);
        else{digitalWrite(LED_RED,LOW); alarmSequenceRunning=false;}
    }
}
}

// Temp/Hum
if(now-lastSwitchMillis>=SWITCH_INTERVAL){
    showTemp=!showTemp;
    lastSwitchMillis=now;
}
float temp=dht.readTemperature();
float hum=dht.readHumidity();
if(!isnan(temp)) lastTemp=temp;
if(!isnan(hum)) lastHum=hum;
lcd.setCursor(0,1);
lcd.print("                ");
lcd.setCursor(0,1);
if(showTemp){lcd.print("Temp: "); lcd.print(lastTemp,1); lcd.print(" C");}
else{lcd.print("Hum:  "); lcd.print(lastHum,0); lcd.print(" %");}

// Abstand
if(now-lastDistanceMillis>=DISTANCE_INTERVAL){
    lastDistanceMillis=now;
    digitalWrite(TRIG_PIN,LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN,HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN,LOW);
    long duration=pulseIn(ECHO_PIN,HIGH,30000);
    if(duration==0) lastDistance=-1;
    else lastDistance=duration*0.0343/2.0;
    lcd.setCursor(0,2);
    lcd.print("                ");
    lcd.setCursor(0,2);
    if(lastDistance<0) lcd.print("Abstand: -- cm");
    else { lcd.print("Abstand: "); lcd.print(lastDistance,1); lcd.print("
cm");}
}
// Ampel
if(!waterAlarm){
    if(lastDistance<4){digitalWrite(LED_GREEN,HIGH);
digitalWrite(LED_YELLOW,LOW); digitalWrite(LED_RED,LOW);}

```

```

        else if(lastDistance<=7){digitalWrite(LED_GREEN,LOW);
digitalWrite(LED_YELLOW,HIGH); digitalWrite(LED_RED,LOW);}
        else if(lastDistance<=12){digitalWrite(LED_GREEN,LOW);
digitalWrite(LED_YELLOW,LOW); digitalWrite(LED_RED,HIGH);}
        else{digitalWrite(LED_GREEN,LOW); digitalWrite(LED_YELLOW,LOW);
digitalWrite(LED_RED,LOW);}
    }
}

// LCD Zeile 4 Sprüche Rotation
if(!waterAlarm && !showAlarmMessage){
    if(now-lastMsgChange>MSG_INTERVAL){
        lastMsgChange=now;
        lcd.setCursor(0,3);
        lcd.print("                ");
        lcd.setCursor(0,3);
        lcd.print(messages[msgIndex]);
        msgIndex++;
        if(msgIndex>=msgCount) msgIndex=0;
    }
}

// Alarm Button Anzeige max 5s
if(showAlarmMessage && now-alarmMsgStart>5000){
    showAlarmMessage=false;
    lcd.setCursor(0,3);
    lcd.print("                ");
}
}

```
